



MEDIATION PRO - USER MANAGEMENT & FRONTEND SPECIFICATION

Complete Design Document for Cursor Implementation

Version: 1.0
Last Updated: January 2025



MỤC LỤC

- 1. [User Management System](#)
- 2. [Database Schema](#)
- 3. [API Endpoints](#)
- 4. [Frontend Screens](#)
- 5. [Stitch Prompts](#)
- 6. [Implementation Guide](#)

1. USER MANAGEMENT SYSTEM

1.1 Tổng quan

Authentication Methods

Method	Mô tả	Use Case
Username/Password	Đăng nhập truyền thống	Internal users, Admin
Google OAuth	Đăng nhập qua Gmail	Users có quyền AdMob

User Roles

Role	Permissions
Super Admin	Quản lý users, full access
Admin	Quản lý users (trừ Super Admin), full access data

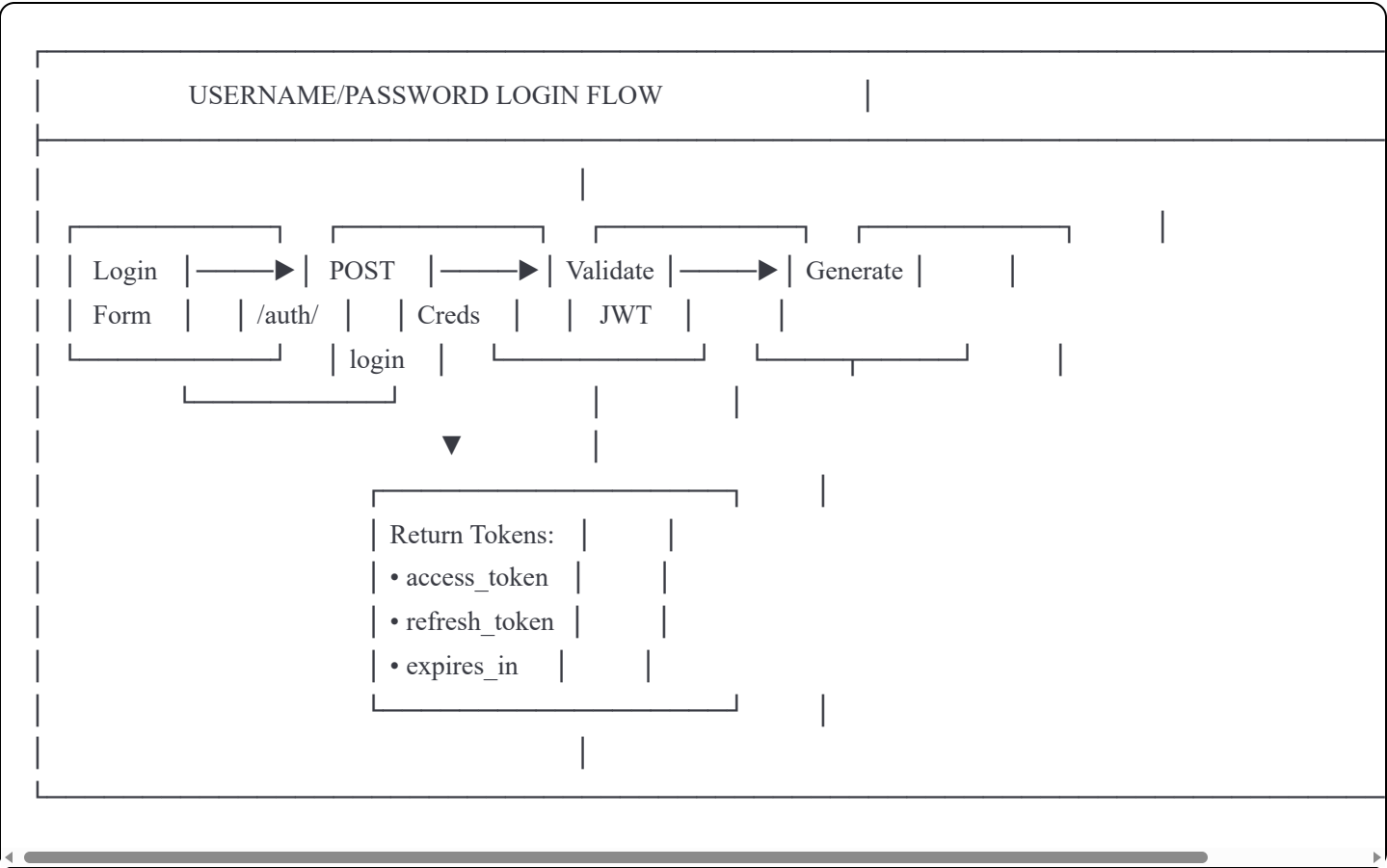
Role	Permissions
Manager	View all, edit recommendations, manage alerts
Viewer	View only, không được edit

Key Rules

- ❌ KHÔNG cho phép self-registration
- ✅ Chỉ Admin tạo được user mới
- ✅ User có thể đổi password của mình
- ✅ Admin có thể reset password của user khác

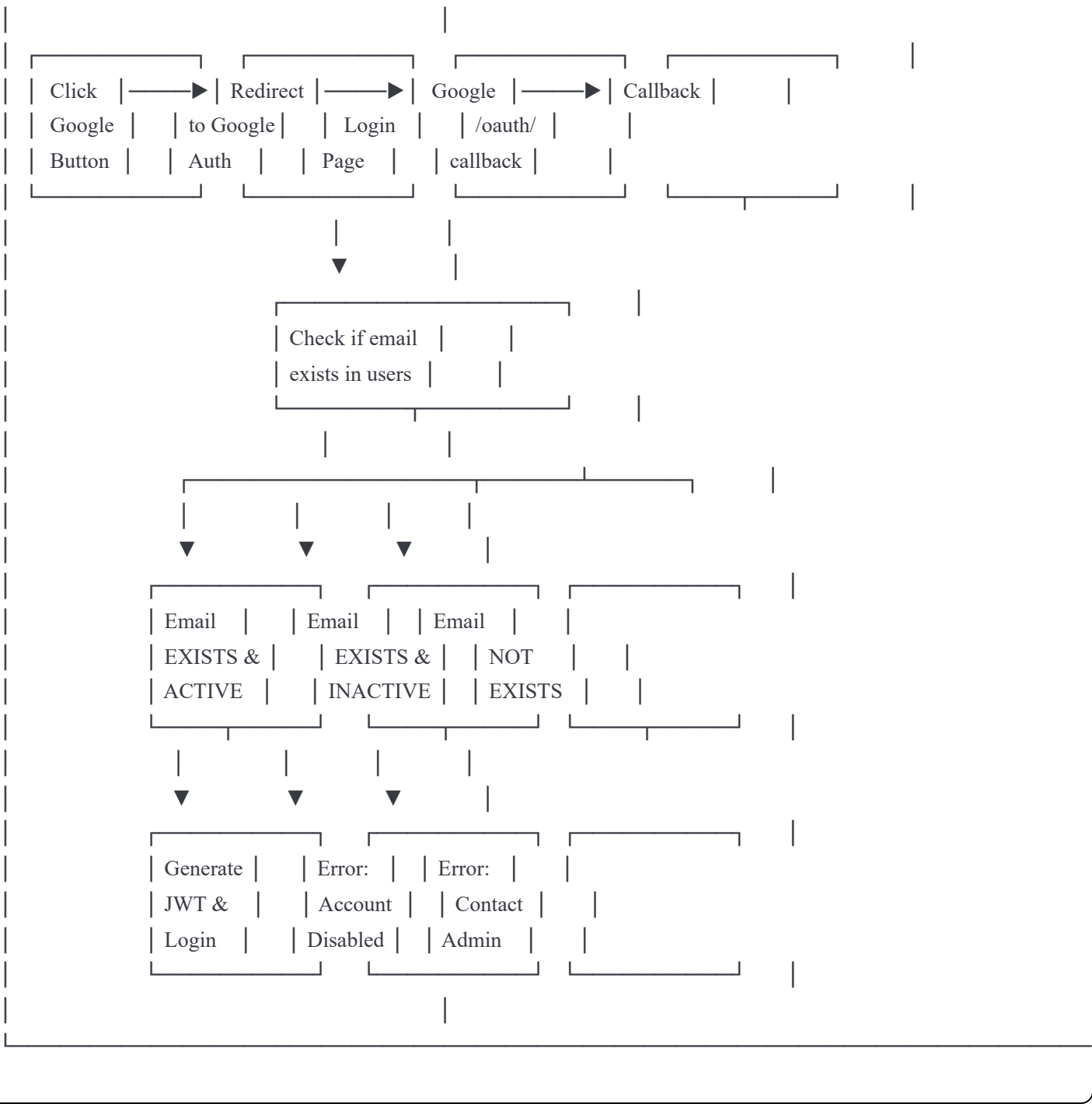
1.2 Authentication Flow

Username/Password Flow



Google OAuth Flow





1.3 Session Management

JWT Token Structure:

```
{
  "sub": "user_id",
  "email": "user@example.com",
  "name": "User Name",
  "role": "admin",
  "permissions": ["view", "edit", "delete"],
  "exp": 1234567890,
  "iat": 1234567800
}
```

```
}
```

Token Expiry:

- Access Token: 1 hour
- Refresh Token: 7 days

Auto Refresh:

- Frontend tự động refresh token khi còn < 5 phút
- Nếu refresh token hết hạn $\rightarrow$ redirect to login

2. DATABASE SCHEMA

2.1 Users Table

```
sql
```

```

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Basic Info
  email VARCHAR(255) UNIQUE NOT NULL,
  username VARCHAR(100) UNIQUE,
  password_hash VARCHAR(255),          -- NULL if OAuth only

  -- Profile
  full_name VARCHAR(255) NOT NULL,
  avatar_url VARCHAR(500),
  phone VARCHAR(20),

  -- Auth Settings
  auth_provider VARCHAR(20) DEFAULT 'local', -- local, google, both
  google_id VARCHAR(100),

  -- Role & Permissions
  role VARCHAR(20) NOT NULL DEFAULT 'viewer', -- super_admin, admin, manager, viewer
  permissions JSONB DEFAULT '[]',

  -- Status
  is_active BOOLEAN DEFAULT true,
  is_email_verified BOOLEAN DEFAULT false,

  -- Security
  failed_login_attempts INTEGER DEFAULT 0,
  locked_until TIMESTAMP WITH TIME ZONE,
  password_changed_at TIMESTAMP WITH TIME ZONE,
  must_change_password BOOLEAN DEFAULT false,

  -- Metadata
  last_login_at TIMESTAMP WITH TIME ZONE,
  last_login_ip VARCHAR(45),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  created_by UUID REFERENCES users(id),

  CONSTRAINT valid_role CHECK (role IN ('super_admin', 'admin', 'manager', 'viewer')),
  CONSTRAINT valid_auth_provider CHECK (auth_provider IN ('local', 'google', 'both'))
);

-- Indexes

```

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_active ON users(is_active);
```

2.2 Refresh Tokens Table

sql

```
CREATE TABLE refresh_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash VARCHAR(255) NOT NULL,
  device_info VARCHAR(500),
  ip_address VARCHAR(45),
  expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  revoked_at TIMESTAMP WITH TIME ZONE,

  CONSTRAINT unique_active_token UNIQUE (token_hash)
);

CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id);
CREATE INDEX idx_refresh_tokens_expires ON refresh_tokens(expires_at);
```

2.3 User Activity Log Table

sql

```
CREATE TABLE user_activity_logs (  
  id BIGSERIAL PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  action VARCHAR(50) NOT NULL,      -- login, logout, password_change, etc.  
  entity_type VARCHAR(50),          -- user, alert, recommendation, etc.  
  entity_id VARCHAR(100),  
  details JSONB,  
  ip_address VARCHAR(45),  
  user_agent VARCHAR(500),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

```
CREATE INDEX idx_activity_user ON user_activity_logs(user_id);  
CREATE INDEX idx_activity_action ON user_activity_logs(action);  
CREATE INDEX idx_activity_created ON user_activity_logs(created_at DESC);
```

2.4 Initial Data

```
sql  
  
-- Create Super Admin  
INSERT INTO users (email, username, password_hash, full_name, role, auth_provider, is_active, is_email_verified)  
VALUES (  
  'admin@mediation.pro',  
  'admin',  
  '$2b$12$LQv3c1yqBWVHxkd0LHAKCOYz6TtxMQJqhN8/X4.VTtYR9Xz6eFGGi', -- "admin123"  
  'System Administrator',  
  'super_admin',  
  'local',  
  true,  
  true  
);
```

3. API ENDPOINTS

3.1 Authentication APIs

POST /api/auth/login

Username/Password Login

http

POST /api/auth/login

Content-Type: application/json

```
{
  "username": "admin",      // or email
  "password": "admin123"
}
```

Response 200:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2gg...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "id": "uuid",
    "email": "admin@mediation.pro",
    "fullName": "System Administrator",
    "role": "super_admin",
    "avatarUrl": null
  }
}
```

Response 401:

```
{
  "error": "invalid_credentials",
  "message": "Invalid username or password"
}
```

Response 423:

```
{
  "error": "account_locked",
  "message": "Account is locked. Try again in 30 minutes"
}
```

GET /api/auth/google

Initiate Google OAuth

http

GET /api/auth/google?redirect_uri=http://localhost:3000/oauth/callback

Response 302:

Redirect to: <https://accounts.google.com/o/oauth2/v2/auth?...>

POST /api/auth/google/callback

Google OAuth Callback

http

POST /api/auth/google/callback

Content-Type: application/json

```
{
  "code": "4/0AX4XfWh...",
  "redirect_uri": "http://localhost:3000/oauth/callback"
}
```

Response 200:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2gg...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": { ... }
}
```

Response 403:

```
{
  "error": "user_not_registered",
  "message": "Your email is not registered. Please contact administrator."
}
```

POST /api/auth/refresh

Refresh Access Token

http

POST /api/auth/refresh

Content-Type: application/json

```
{
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2gg..."
}
```

Response 200:

```
{
  "access_token": "new_access_token...",
  "expires_in": 3600
}
```

POST /api/auth/logout

Logout & Revoke Token

http

POST /api/auth/logout

Authorization: Bearer {access_token}

Response 200:

```
{
  "message": "Logged out successfully"
}
```

3.2 User Profile APIs

GET /api/users/me

Get Current User Profile

http

GET /api/users/me

Authorization: Bearer {access_token}

Response 200:

```
{
  "id": "uuid",
  "email": "user@example.com",
  "username": "username",
  "fullName": "Full Name",
  "avatarUrl": "https://...",
  "phone": "+84123456789",
  "role": "admin",
  "authProvider": "both",
  "lastLoginAt": "2025-01-15T10:00:00Z",
  "createdAt": "2024-12-01T00:00:00Z"
}
```

PUT /api/users/me

Update Current User Profile

http

PUT /api/users/me

Authorization: Bearer {access_token}

Content-Type: application/json

```
{
  "fullName": "New Name",
  "phone": "+84987654321",
  "avatarUrl": "https://..."
}
```

Response 200:

```
{
  "message": "Profile updated successfully",
  "user": { ... }
}
```

PUT /api/users/me/password

Change Own Password

http

PUT /api/users/me/password

Authorization: Bearer {access_token}

Content-Type: application/json

```
{
  "currentPassword": "old_password",
  "newPassword": "new_password",
  "confirmPassword": "new_password"
}
```

Response 200:

```
{
  "message": "Password changed successfully"
}
```

Response 400:

```
{
  "error": "invalid_password",
  "message": "Current password is incorrect"
}
```

3.3 User Management APIs (Admin Only)

GET /api/admin/users

List All Users

http

GET /api/admin/users?page=1&pageSize=20&role=admin&search=john

Authorization: Bearer {access_token}

Response 200:

```
{
  "users": [
    {
      "id": "uuid",
      "email": "user@example.com",
      "username": "username",
      "fullName": "Full Name",
      "role": "admin",
      "isActive": true,
      "lastLoginAt": "2025-01-15T10:00:00Z",
      "createdAt": "2024-12-01T00:00:00Z"
    }
  ],
  "total": 25,
  "page": 1,
  "pageSize": 20
}
```

POST /api/admin/users

Create New User

http

POST /api/admin/users

Authorization: Bearer {access_token}

Content-Type: application/json

```
{
  "email": "newuser@example.com",
  "username": "newuser",
  "fullName": "New User",
  "role": "manager",
  "authProvider": "both",
  "password": "initial_password", // Optional, generate if not provided
  "sendWelcomeEmail": true
}
```

Response 201:

```
{
  "message": "User created successfully",
  "user": { ... },
  "temporaryPassword": "abc123xyz" // Only if password not provided
}
```

GET /api/admin/users/{id}

Get User Details

http

GET /api/admin/users/{id}

Authorization: Bearer {access_token}

Response 200:

```
{
  "id": "uuid",
  "email": "user@example.com",
  "username": "username",
  "fullName": "Full Name",
  "phone": "+84123456789",
  "role": "admin",
  "authProvider": "both",
  "isActive": true,
  "isEmailVerified": true,
  "lastLoginAt": "2025-01-15T10:00:00Z",
  "lastLoginIp": "192.168.1.1",
  "createdAt": "2024-12-01T00:00:00Z",
  "createdBy": {
    "id": "uuid",
    "fullName": "Admin Name"
  }
}
```

PUT /api/admin/users/{id}

Update User

http

PUT /api/admin/users/{id}

Authorization: Bearer {access_token}

Content-Type: application/json

```
{
  "fullName": "Updated Name",
  "role": "viewer",
  "isActive": true
}
```

Response 200:

```
{
  "message": "User updated successfully",
  "user": { ... }
}
```

DELETE /api/admin/users/{id}

Delete User (Soft Delete)

http

DELETE /api/admin/users/{id}

Authorization: Bearer {access_token}

Response 200:

```
{
  "message": "User deleted successfully"
}
```

Response 403:

```
{
  "error": "forbidden",
  "message": "Cannot delete super admin user"
}
```

POST /api/admin/users/{id}/reset-password

Reset User Password

http

POST /api/admin/users/{id}/reset-password

Authorization: Bearer {access_token}

Content-Type: application/json

```
{
  "sendEmail": true,
  "requireChange": true
}
```

Response 200:

```
{
  "message": "Password reset successfully",
  "temporaryPassword": "temp123abc"
}
```

POST /api/admin/users/{id}/toggle-status

Enable/Disable User

http

POST /api/admin/users/{id}/toggle-status

Authorization: Bearer {access_token}

Response 200:

```
{
  "message": "User status updated",
  "isActive": false
}
```

3.4 Notifications API

GET /api/notifications

Get User Notifications (Alerts)

http

GET /api/notifications?page=1&pageSize=10&unreadOnly=true

Authorization: Bearer {access_token}

Response 200:

```
{
  "notifications": [
    {
      "id": 123,
      "type": "alert",
      "title": "Revenue Drop Alert",
      "message": "Daily revenue dropped -18.5%",
      "priority": "HIGH",
      "alertId": 456,
      "appId": "ca-app-pub-xxx",
      "appName": "WeatherPlus Pro",
      "isRead": false,
      "createdAt": "2025-01-15T09:15:00Z",
      "data": {
        "ruleCode": "REV-001",
        "metricValue": -18.5,
        "threshold": -10
      }
    }
  ],
  "total": 45,
  "unreadCount": 12
}
```

GET /api/notifications/{id}

Get Notification Detail

http

GET /api/notifications/{id}

Authorization: Bearer {access_token}

Response 200:

```
{
  "id": 123,
  "type": "alert",
  "title": "Revenue Drop Alert",
  "message": "Daily revenue dropped -18.5%",
  "priority": "HIGH",
  "alertId": 456,
  "appId": "ca-app-pub-xxx",
  "appName": "WeatherPlus Pro",
  "mediationGroupId": "mg-123",
  "mediationGroupName": "Banner US",
  "isRead": true,
  "createdAt": "2025-01-15T09:15:00Z",
  "data": {
    "ruleCode": "REV-001",
    "ruleName": "Daily Revenue Drop",
    "metricValue": -18.5,
    "threshold": -10,
    "previousValue": 1245.00,
    "currentValue": 1015.30,
    "recommendations": [
      "Check eCPM changes in top waterfall lines",
      "Review fill rate by ad source"
    ]
  }
}
```

POST /api/notifications/{id}/read

Mark as Read

http

POST /api/notifications/{id}/read

Authorization: Bearer {access_token}

Response 200:

```
{  
  "message": "Marked as read"  
}
```

POST /api/notifications/read-all

Mark All as Read

http

POST /api/notifications/read-all

Authorization: Bearer {access_token}

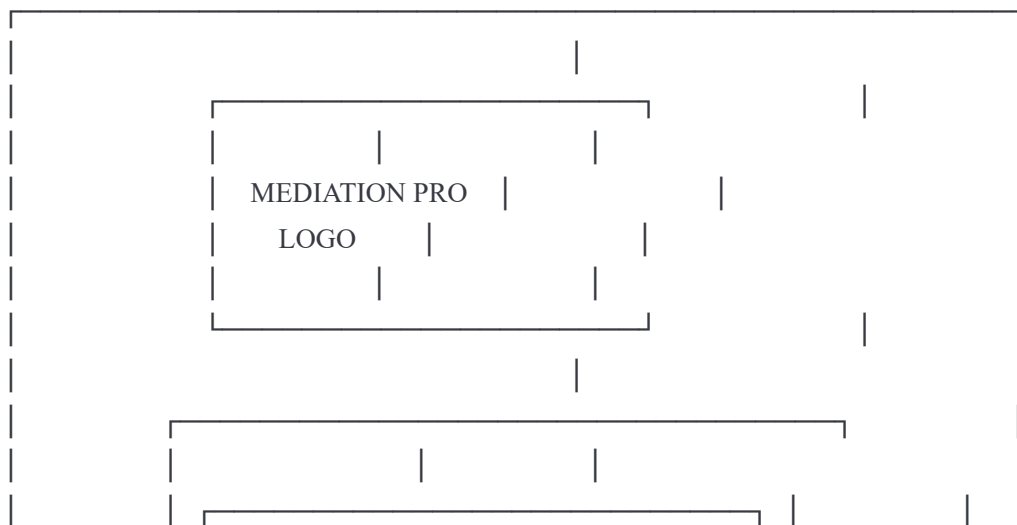
Response 200:

```
{  
  "message": "All notifications marked as read",  
  "count": 12  
}
```

4. FRONTEND SCREENS

4.1 Login Screen

Layout





Search users...

Role: [All ▼]

Status: [All ▼]



User

Email

Role

Status



John Doe

john@company.com

Admin



Active



Jane Smith

jane@company.com

Manager



Active



Bob Wilson

bob@company.com

Viewer



Inactive



Alice Johnson

alice@company.com

Admin



Active

Showing 1-4 of 25 users

[< Prev] 1 2 3 [Next >]

Action Menu (:):



View Details



Edit



Reset Password



Disable

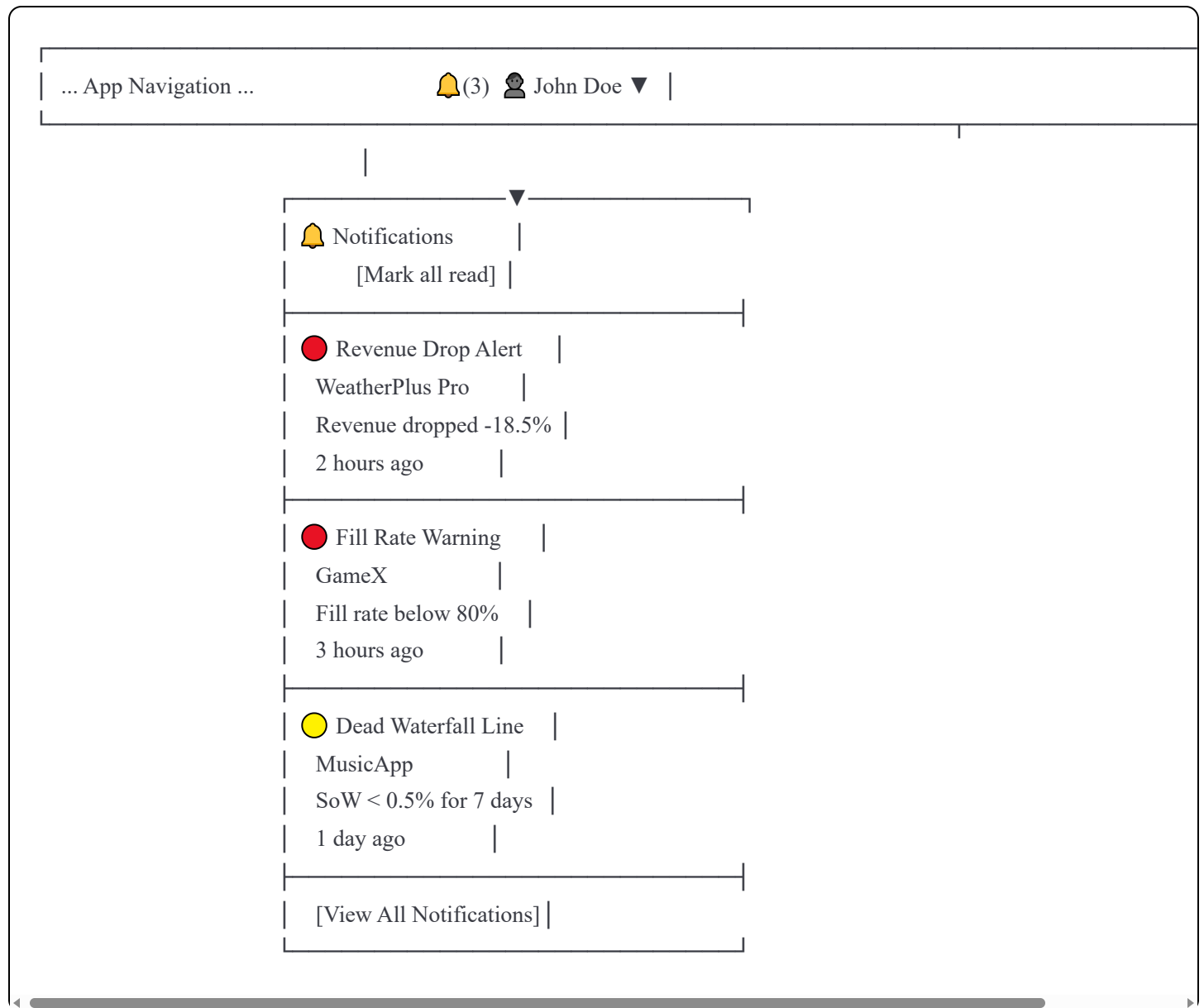


Delete

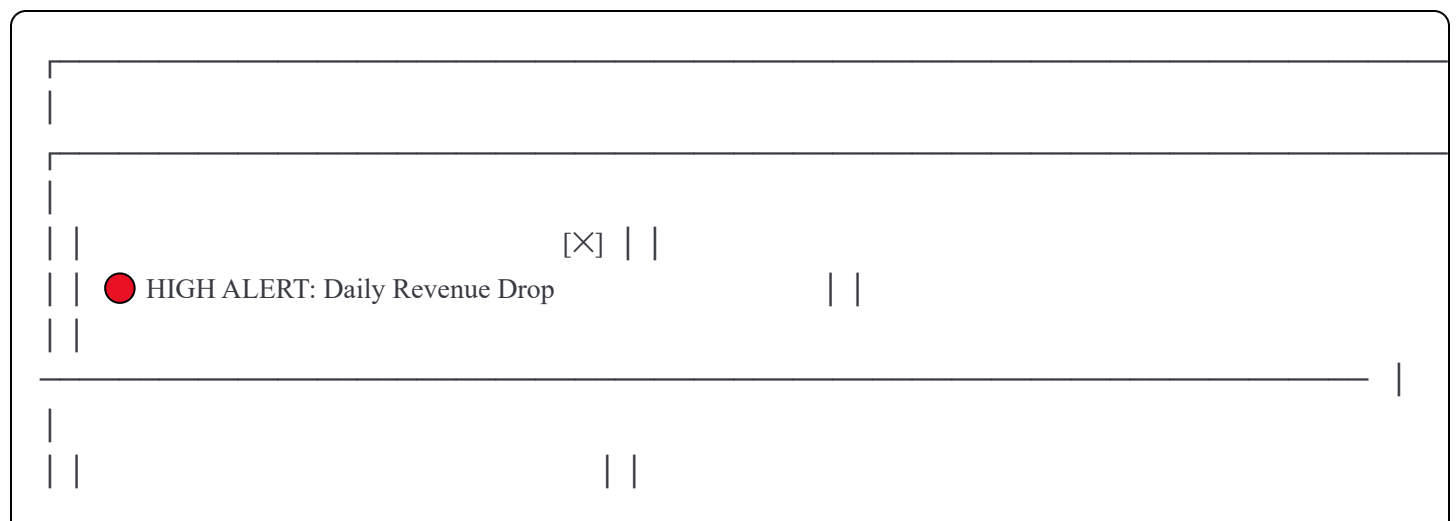
4.3 Add/Edit User Modal

4.5 Notification Panel

Notification Bell Dropdown



Notification Detail Popup





App: WeatherPlus Pro

ca-app-pub-1234567890~9876543210



Triggered: January 15, 2025 at 09:15 UTC



Issue

Daily revenue dropped -18.5%



Metrics

• Yesterday: \$1,245.00

• Today: \$1,015.30

• Change: -\$229.70 (-18.5%)



Affected Mediation Groups

• Banner US (-22%)

• Interstitial Global (-15%)



Recommended Actions

1. Check eCPM changes in top waterfall lines
2. Review fill rate by ad source
3. Verify no technical issues with SDK



[View in Dashboard]

[✓ Acknowledge]

[× Close]

4.6 My Profile Screen

MEDIATION PRO

Dashboard | Apps | ...

 Admin ▼

 My Profile



John Doe

Avatar

john@company.com

Admin • Member since Dec 2024

 Change

 Personal Information

 Security

Full Name

John Doe

Password

 Change Password

Last changed: 30 days ago

Two-Factor Auth

Not configured

[Enable 2FA]

Email

john@company.com

Connected Accounts

☒ Google (john@gmail.com)

Phone

Sessions

+84 123 456 789

3 active sessions

[Manage Sessions]

Save Changes

5. STITCH PROMPTS

5.1 Login Screen Prompt

Create a modern login screen for Mediation Pro with the following requirements:

LAYOUT:

- Centered card on a gradient blue background

- Logo at top of card

- Clean, minimal design with good spacing

FORM ELEMENTS:

1. Email/Username input field with icon

2. Password input field with show/hide toggle icon

3. "Remember me" checkbox

4. "Forgot password?" link

5. Primary "Sign In" button (blue, full width)

6. Divider with "OR" text

7. "Sign in with Google" button (white with Google logo, red accent)

8. Footer text: "Need access? Contact administrator"

STATES:

- Default: Empty form

- Hover: Button color changes

- Focus: Input border highlight

- Loading: Button shows spinner, disabled

- Error: Red border on invalid fields, error message below form

- Success: Brief green checkmark before redirect

VALIDATION:

- Email/username required

- Password required (min 6 characters)

- Show inline validation errors

RESPONSIVE:

- Mobile: Full width card with padding
- Desktop: Max width 400px, centered

COLORS:

- Primary: #1976d2 (blue)
- Google button: #db4437 (red)
- Error: #f44336
- Background gradient: #1565c0 to #0d47a1

5.2 User Management Screen Prompt

Create an admin user management screen with the following requirements:

HEADER:

- Page title "User Management" with user icon
- "Add User" button on the right (primary blue)

FILTERS BAR:

- Search input (search by name, email)
- Role dropdown filter (All, Super Admin, Admin, Manager, Viewer)
- Status dropdown filter (All, Active, Inactive)

DATA TABLE:

- Columns: Checkbox, User (avatar + name), Email, Role, Status, Last Login, Actions
- Avatar: Circle with initials if no image
- Status badge: Green dot for Active, Red dot for Inactive
- Role: Colored badge (Admin=Blue, Manager=Purple, Viewer=Gray)
- Actions: Three-dot menu

TABLE FEATURES:

- Sortable columns
- Bulk selection checkbox
- Row hover highlight
- Pagination at bottom

ACTION MENU (on click three-dot):

- View Details
- Edit
- Reset Password
- Disable/Enable (toggle based on status)

- Delete (red color, with confirmation)

EMPTY STATE:

- Icon and message when no users match filter

RESPONSIVE:

- Mobile: Card view instead of table
- Tablet: Horizontal scroll for table

5.3 Add/Edit User Modal Prompt

Create a modal dialog for adding/editing users:

MODAL:

- Max width 500px
- Title: "Add New User" or "Edit User"
- Close button (X) in top right

FORM FIELDS:

1. Email (required) - text input
2. Username (optional) - text input
3. Full Name (required) - text input
4. Phone (optional) - phone input with country code
5. Role (required) - select dropdown with options:
 - Viewer (default)
 - Manager
 - Admin
 - Super Admin (only visible if current user is Super Admin)
6. Authentication Method - radio buttons:
 - Username/Password only
 - Google OAuth only
 - Both (default)
7. Checkboxes:
 - Send welcome email with credentials (checked by default)
 - Require password change on first login

FOOTER:

- Cancel button (outline/secondary)
- Save button (primary blue)

VALIDATION:

- Email: Required, valid email format, unique check
- Username: If provided, alphanumeric, 3-20 chars, unique check

- Full Name: Required, 2-100 chars
- Real-time validation with inline errors

LOADING STATE:

- Spinner overlay while saving
- Disable all inputs and buttons

5.4 User Profile Dropdown Prompt

Create a user profile dropdown menu that appears when clicking the user avatar/name in the header:

TRIGGER:

- User avatar (small circle) + Name + dropdown arrow
- Click to open dropdown

DROPDOWN CONTENT:

1. Header section:

- Larger avatar
- User's full name (bold)
- Email (smaller, gray)
- Role badge

2. Menu items (with icons):

- My Profile (user icon)
- Change Password (key icon)
- Settings (gear icon)
- Divider line
- Sign Out (exit icon, slightly red tint)

BEHAVIOR:

- Opens on click (not hover)
- Closes when clicking outside
- Closes when selecting an item
- Smooth open/close animation
- Position: Below trigger, right-aligned

STYLING:

- White background
- Subtle shadow
- Rounded corners
- Icons in gray, text in dark gray

- Hover state: Light gray background
- Sign Out: Red tint on hover

5.5 Notification Panel Prompt

Create a notification bell with dropdown panel:

NOTIFICATION BELL (in header):

- Bell icon
- Red badge with count (e.g., "3") if unread notifications
- Badge hidden if count is 0
- Click to open panel

NOTIFICATION PANEL:

- Width: 360px
- Max height: 480px with scroll
- Position: Below bell, right-aligned

PANEL HEADER:

- Title "Notifications"
- "Mark all as read" link button

NOTIFICATION ITEMS:

Each item shows:

- Priority indicator (left border color: red=HIGH, yellow=MEDIUM, green=LOW)
- Title (bold, truncate if long)
- App name (smaller, gray)
- Brief message (2 lines max)
- Time ago (e.g., "2 hours ago")
- Unread indicator (blue dot)

ITEM BEHAVIOR:

- Hover: Light background
- Click: Opens detail popup
- Swipe left (mobile): Mark as read

PANEL FOOTER:

- "View All Notifications" link to full page

EMPTY STATE:

- Bell icon

- "No notifications"
- "You're all caught up!"

5.6 Notification Detail Popup Prompt

Create a modal popup for notification/alert details:

MODAL:

- Max width 600px
- Overlay background (dark semi-transparent)
- Close button (X) in top right

HEADER:

- Priority badge with icon ( HIGH ALERT /  WARNING /  INFO)
- Alert type/name (e.g., "Daily Revenue Drop")

CONTENT SECTIONS:

1. Context:

- App name and ID
- Triggered timestamp

2. Issue Card (light gray background):

- Issue description
- Metrics section with before/after comparison:
 - Yesterday: \$1,245.00
 - Today: \$1,015.30
 - Change: -\$229.70 (-18.5%)
- Affected items list (if applicable)

3. Recommended Actions:

- Numbered list of suggested actions

FOOTER:

- "View in Dashboard" button (outline, links to relevant page)
- "Acknowledge" button (primary, marks as acknowledged)
- "Close" button (secondary)

ANIMATION:

- Fade in overlay
- Slide up modal
- Close: reverse animation

5.7 My Profile Screen Prompt

Create a user profile management screen:

PAGE HEADER:

- Title "My Profile" with user icon
- Breadcrumb: Home > My Profile

LAYOUT: Two columns on desktop, stacked on mobile

LEFT COLUMN - Profile Card:

- Large avatar with camera icon overlay for changing
- Name, email, role
- Member since date

LEFT COLUMN - Personal Information:

- Full Name input
- Email input (disabled/locked icon, can't change)
- Phone input
- Save Changes button

RIGHT COLUMN - Security:

1. Password section:

- Last changed date
- "Change Password" button (opens modal)

2. Two-Factor Auth (future feature):

- Status indicator
- Enable/Disable toggle

3. Connected Accounts:

- Google account status
- Connect/Disconnect option

4. Active Sessions:

- Count of active sessions
- "Manage Sessions" link

CHANGE PASSWORD MODAL:

- Current Password input
- New Password input with strength indicator
- Confirm Password input

- Cancel and Save buttons
- Password requirements list

6. IMPLEMENTATION GUIDE

6.1 Frontend Tech Stack

React 18 + TypeScript

- └─ State Management: Zustand or Redux Toolkit
- └─ Routing: React Router v6
- └─ UI Components: TailwindCSS + Headless UI
- └─ Forms: React Hook Form + Zod
- └─ HTTP Client: Axios
- └─ Auth: JWT tokens in HttpOnly cookies
- └─ Real-time: WebSocket for notifications

6.2 Key React Components

```
src/
├── components/
│   ├── auth/
│   │   ├── LoginForm.tsx
│   │   ├── GoogleLoginButton.tsx
│   │   └── ProtectedRoute.tsx
│   ├── layout/
│   │   ├── Header.tsx
│   │   ├── UserDropdown.tsx
│   │   └── NotificationPanel.tsx
│   ├── users/
│   │   ├── UserTable.tsx
│   │   ├── UserModal.tsx
│   │   └── UserFilters.tsx
│   ├── notifications/
│   │   ├── NotificationBell.tsx
│   │   ├── NotificationList.tsx
│   │   └── NotificationDetailModal.tsx
│   └── profile/
│       ├── ProfileCard.tsx
│       ├── PersonalInfoForm.tsx
│       └── ChangePasswordModal.tsx
```

```
├── hooks/
│   ├── useAuth.ts
│   ├── useNotifications.ts
│   └── useUsers.ts
├── services/
│   ├── authService.ts
│   ├── userService.ts
│   └── notificationService.ts
├── stores/
│   ├── authStore.ts
│   └── notificationStore.ts
└── pages/
    ├── LoginPage.tsx
    ├── DashboardPage.tsx
    ├── UsersPage.tsx
    └── ProfilePage.tsx
```

6.3 Auth Context Example

typescript

```
// stores/authStore.ts
```

```
import { create } from 'zustand';
```

```
import { persist } from 'zustand/middleware';
```

```
interface User {
```

```
  id: string;
```

```
  email: string;
```

```
  fullName: string;
```

```
  role: string;
```

```
  avatarUrl?: string;
```

```
}
```

```
interface AuthState {
```

```
  user: User | null;
```

```
  accessToken: string | null;
```

```
  isAuthenticated: boolean;
```

```
  isLoading: boolean;
```

```
  login: (email: string, password: string) => Promise<void>;
```

```
  loginWithGoogle: (code: string) => Promise<void>;
```

```
  logout: () => Promise<void>;
```

```
  refreshToken: () => Promise<void>;
```

```
}
```

```
export const useAuthStore = create<AuthState>()(
```

```
  persist(
```

```
    (set, get) => ({
```

```
      user: null,
```

```
      accessToken: null,
```

```
      isAuthenticated: false,
```

```
      isLoading: false,
```

```
      login: async (email, password) => {
```

```
        set({ isLoading: true });
```

```
        try {
```

```
          const response = await authService.login(email, password);
```

```
          set({
```

```
            user: response.user,
```

```
            accessToken: response.access_token,
```

```
            isAuthenticated: true,
```

```
          });
```

```
        } finally {
```

```
          set({ isLoading: false });
```

```

    }
  },

  loginWithGoogle: async (code) => {
    set({ isLoading: true });
    try {
      const response = await authService.googleCallback(code);
      set({
        user: response.user,
        accessToken: response.access_token,
        isAuthenticated: true,
      });
    } finally {
      set({ isLoading: false });
    }
  },

  logout: async () => {
    await authService.logout();
    set({ user: null, accessToken: null, isAuthenticated: false });
  },

  refreshToken: async () => {
    try {
      const response = await authService.refresh();
      set({ accessToken: response.access_token });
    } catch {
      get().logout();
    }
  },
},
{ name: 'auth-storage' }
)
);

```

6.4 API Service Example

typescript

```
// services/userService.ts
```

```
import api from './api';
```

```
export const userService = {
```

```
  // Admin APIs
```

```
  getUsers: async (params: { page?: number; pageSize?: number; role?: string; search?: string }) => {  
    const response = await api.get('/admin/users', { params });  
    return response.data;  
  },
```

```
  createUser: async (data: CreateUserDto) => {  
    const response = await api.post('/admin/users', data);  
    return response.data;  
  },
```

```
  updateUser: async (id: string, data: UpdateUserDto) => {  
    const response = await api.put(`/admin/users/${id}`, data);  
    return response.data;  
  },
```

```
  deleteUser: async (id: string) => {  
    const response = await api.delete(`/admin/users/${id}`);  
    return response.data;  
  },
```

```
  resetPassword: async (id: string, options: { sendEmail?: boolean; requireChange?: boolean }) => {  
    const response = await api.post(`/admin/users/${id}/reset-password`, options);  
    return response.data;  
  },
```

```
  toggleStatus: async (id: string) => {  
    const response = await api.post(`/admin/users/${id}/toggle-status`);  
    return response.data;  
  },
```

```
  // Profile APIs
```

```
  getProfile: async () => {  
    const response = await api.get('/users/me');  
    return response.data;  
  },
```

```
  updateProfile: async (data: UpdateProfileDto) => {  
    const response = await api.put('/users/me', data);
```

```
    return response.data;
  },

  changePassword: async (data: ChangePasswordDto) => {
    const response = await api.put('/users/me/password', data);
    return response.data;
  },
};
```

6.5 Protected Route Example

typescript


```

// components/auth/ProtectedRoute.tsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAuthStore } from '@stores/authStore';

interface ProtectedRouteProps {
  children: React.ReactNode;
  requiredRoles?: string[];
}

export function ProtectedRoute({ children, requiredRoles }: ProtectedRouteProps) {
  const { isAuthenticated, user, isLoading } = useAuthStore();
  const location = useLocation();

  if (isLoading) {
    return <LoadingSpinner />;
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (requiredRoles && user && !requiredRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return <>{children}</>;
}

// Usage in routes:
<Route
  path="/admin/users"
  element={
    <ProtectedRoute requiredRoles={['super_admin', 'admin']>
      <UsersPage />
    </ProtectedRoute>
  }
/>

```

SUMMARY CHECKLIST

Screens to Create

- ☐ Login Screen (Username/Password + Google OAuth)
- ☐ User Management Screen (Admin only)
- ☐ Add/Edit User Modal
- ☐ My Profile Screen
- ☐ Change Password Modal

Components to Add to Existing Screens

- ☐ User Profile Dropdown in Header
- ☐ Notification Bell with Panel
- ☐ Notification Detail Popup

Backend APIs Needed

- ☐ POST /api/auth/login
- ☐ GET /api/auth/google
- ☐ POST /api/auth/google/callback
- ☐ POST /api/auth/refresh
- ☐ POST /api/auth/logout
- ☐ GET /api/users/me
- ☐ PUT /api/users/me
- ☐ PUT /api/users/me/password
- ☐ GET /api/admin/users
- ☐ POST /api/admin/users
- ☐ PUT /api/admin/users/{id}
- ☐ DELETE /api/admin/users/{id}
- ☐ POST /api/admin/users/{id}/reset-password
- ☐ POST /api/admin/users/{id}/toggle-status
- ☐ GET /api/notifications
- ☐ GET /api/notifications/{id}
- ☐ POST /api/notifications/{id}/read
- ☐ POST /api/notifications/read-all

Database Tables

- ☐ users
 - ☐ refresh_tokens
 - ☐ user_activity_logs
-

